skmprabhu252 / nfs-benchmark-suite

Type / to search

Code Issues Pull requests Actions Projects Security and quality Insights Settings

On April 24 we'll start using GitHub Copilot interaction data for AI model training unless you opt out. Review this update and manage your preferences in your GitHub account settings.

nfs-benchmark-suite

Private

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 1

skmprabhu252

Languages

- Python 96.7%
- Shell 3.3%

Suggested workflows

Based on your tech stack

Python application

Create and test a Python application.

Configure

SLSA Generic generator

Generate SLSA3 provenance for your existing release workflows

Configure

Python package

Create and test a Python package on multiple Python versions.

Configure

More workflows

Dismiss suggestions

m... 1 Branch 0 Tags Go to file t Go to file Add file Code

skmprabhu3 Update benchmark suite documentation and long test configuration

4f967ec · 29 minutes ago 67 Commits

config	Update benchmark suite documentation ...	29 minutes ago

lib	Force --skip-cleanup flag for NFS backen...	2 hours ago
.gitignore	Initial commit: NFS Benchmark Suite	yesterday
README.md	Update benchmark suite documentation ...	29 minutes ago
generate_html_report.py	Fix: Create HTML reports in report/ direct...	19 hours ago
requirements.txt	docs: Replace 'Performance Test' with 'B...	20 hours ago
runtest.py	Fix DD cleanup error: use file1/file2 instea...	2 hours ago
setup_and_verify.sh	Add libnsl dependency for dbench and re...	3 hours ago

NFS Benchmark Suite

A comprehensive benchmarking tool for evaluating NFS storage performance using industry-standard tools (DD, FIO, IOzone, Bonnie++, dbench).

Table of Contents

1. Why Use This Tool?
2. Features
3. Architecture
4. Test Profiles
5. Performance Dimensions
6. Requirements
7. Installation

8. Quick Start
9. Usage
10. Configuration
11. Understanding Results
12. Best Practices
13. Troubleshooting
14. Author

Why Use This Tool?

This tool helps you **measure and validate NFS storage performance** through comprehensive benchmarking:

When to Use:

- **Before Production**: Validate NFS performance meets application requirements
- **After Changes**: Verify performance impact of configuration changes (mount options, NFS version, network upgrades)
- **Troubleshooting**: Identify performance bottlenecks (storage, network, or NFS layer)
- **Capacity Planning**: Understand throughput, IOPS, and latency characteristics under load
- **Regression Testing**: Track performance over time and detect degradations

Key Benefits:

- **Comprehensive**: Runs 5 industry-standard benchmarks (DD, FIO, IOzone, Bonnie++, dbench) in one command
- **NFS-Specific**: Collects NFS metrics (RPC stats, transport stats, retransmissions) alongside performance data
- **Baseline Comparison**: Compares results against expected performance for your network speed
- **Automated Analysis**: Identifies bottlenecks and provides actionable recommendations
- **Historical Tracking**: Detects performance regressions by comparing with previous test runs

Features

- **Multiple Benchmarks**: DD, FIO, IOzone, Bonnie++, and dbench for comprehensive testing
- **NFS Metrics**: Collects NFS statistics including transport stats, RPC metrics, and per-operation timing
- **Performance Analysis**: Automatically identifies bottlenecks and suggests improvements
- **Historical Tracking**: Compares results with previous runs to detect performance regressions
- **Industry Baselines**: Shows how your results compare to expected performance

Architecture

High-Level Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                          runtest.py                          │
│                     (Main Test Orchestrator)                 │
└──────────────────────────────────────────────────────────────┘
              │
              │
              ├──────────────────────────────────────────────┐
              │                                              │
              v                                              v
┌──────────────────────────┐                   ┌──────────────────────────┐
│   Input Validation       │                   │   Metrics Collection     │
│  ────────────────        │                   │  ────────────────────    │
│  • Mount Path Check      │                   │  • System Metrics        │
│  • Config Validation     │                   │  • NFS Metrics           │
│  • Space Check           │                   │  • Network Stats         │
│  • Permission Check      │                   │  • xprt Statistics       │
└──────────────────────────┘                   └──────────────────────────┘
           │                                              │
           │                                              │
           v                                              v
┌──────────────────────────────────────────────────────────────┐
│                       Benchmark Modules                      │
```

```
│  ──────────────────────────────────────────────────────────────────────────  │
│  ┌──────────┐   ┌──────────┐   ┌──────────┐   ┌──────────┐                  │
│  │    DD    │   │   FIO    │   │ IOzone   │   │ Bonnie++ │                  │
│  └──────────┘   └──────────┘   └──────────┘   └──────────┘                  │
│  ┌──────────┐                                                                │
│  │ dbench   │   All inherit from BaseTestTool                                │
│  └──────────┘                                                                │
└──────────────────────────────────────────────────────────────────────────────┘
                  │
                  │
                  v
┌──────────────────────────────────────────────────────────────────────────────┐
│                          Analysis & Reporting                                │
│  ──────────────────────────────────────────────────────────────────────────  │
│  • Performance Analyzer (Root Cause Detection)                               │
│  • Historical Comparison (Regression Detection)                              │
│  • HTML Report Generator (Interactive Charts)                                │
│  • Executive Summary (Intelligent Insights)                                  │
└──────────────────────────────────────────────────────────────────────────────┘
```

Data Flow

```
┌──────────────┐
│ User Input   │
│ (CLI Args)   │
└──────────────┘
       │
       v
┌──────────────────────┐
│ Validation Layer     │
│ • Mount Path         │
│ • Configuration      │
│ • Prerequisites      │
└──────────────────────┘
       │
       v
┌──────────────────────┐
│ Test Execution       │
```

```
│  ┌─────────────────┐   │
│  │ Start Metrics   │   │
│  │ Run Test        │   │
│  │ Stop Metrics    │   │
│  │ Collect Results│   │
│  └─────────────────┘   │
└────────────────────────┘
        │
        │
        v
┌────────────────────────┐
│ Analysis Layer         │
│ • Performance          │
│ • Historical           │
│ • Root Cause           │
└────────────────────────┘
        │
        │
        v
┌────────────────────────┐
│ Output Generation      │
│ • JSON Results         │
│ • HTML Report          │
│ • Console Summary      │
└────────────────────────┘
```

- **HTML Reports**: Interactive reports with charts, trends, and executive summary
- **Flexible Testing**: Quick (~15 min) or comprehensive (4-8 hours) test profiles

Test Profiles

Profile Comparison

Aspect	Quick Test	Long Test
Duration	~15 minutes	4-8 hours

Aspect	Quick Test	Long Test
Purpose	Validation & Sanity Check	Production Benchmark
File Sizes	500MB - 2GB	8GB - 64GB
Runtime per Test	30-90 seconds	30-120 minutes
Threads/Clients	2-8	8-128
Total Data Written	~20-30GB	~500GB-1TB
Disk Space Required	50GB min, 100GB rec	1TB min, 2TB rec
Command	`--quick-test`	`--long-test`

Quick Test (~15 minutes)

Purpose: Rapid validation to ensure installation, setup, and environment are working correctly

Command:

```
python3 runtest.py --mount-path /mnt/nfs1 --quick-test
```

Long Test (4-8 hours)

Purpose: Comprehensive production benchmark for detailed performance analysis

Test Duration Breakdown:

Tool	Duration	Purpose
DD Tests	~5 minutes	Sanity check
FIO Tests	~4.5 hours	Sequential, random, mixed, metadata, latency

Tool	Duration	Purpose
IOzone Tests	~1-2 hours	Filesystem operations and scaling
Bonnie++ Tests	~6-8 hours	Comprehensive filesystem stress testing
dbench Tests	~2-3 hours	Multi-client workload simulation

Command:

```
python3 runtest.py --mount-path /mnt/nfs1 --long-test
```

Performance Dimensions

Both test profiles measure these **6 critical performance dimensions**:

Dimension	What It Measures	Why It Matters	Tools Used
1. Throughput (MB/s)	Sequential data transfer rate for large files	Critical for bulk data operations, backups, large file transfers	DD, FIO (sequential), IOzone (baseline), Bonnie++
2. IOPS	Random I/O performance with small block sizes (4K)	Database workloads, virtual machines, application servers	FIO (random 4K), IOzone (random I/O)
3. Latency (ms)	Response time for I/O operations	Interactive applications, real-time systems, user experience	FIO (latency test), dbench (single client)
4. Metadata ops/sec	File creation, deletion, stat, rename operations	Applications with many small files, build systems, source code repos	FIO (metadata), IOzone, Bonnie++ (file ops), dbench
5. Cache Effects	Performance difference between cached and direct I/O	Understanding client-side caching behavior, tuning cache parameters	DD (cached vs direct), IOzone (cache behavior), FIO

Dimension	What It Measures	Why It Matters	Tools Used
6. Concurrency Scaling	Performance scaling with multiple concurrent clients/threads	Multi-user environments, parallel workloads, capacity planning	IOzone (scaling), FIO (numjobs), dbench (scalability)

Requirements

- **System**: Linux (kernel 4.x+), Python 3.6+
- **Disk Space**:
  - Quick Test: 50GB minimum, 100GB recommended
  - Long Test: 1TB minimum, 2TB recommended
- **Network**: 1 Gbps or higher recommended
- **Tools**: fio, iozone, bonnie++, dbench, nfs-common

Installing Prerequisites

Ubuntu/Debian

```
sudo apt-get update
sudo apt-get install python3 python3-pip fio iozone bonnie++ dbench libnsl2 nfs-common
```

RHEL/CentOS/Fedora

```
# Install core packages (always available)
sudo dnf install -y python3 python3-pip fio nfs-utils libnsl

# Optional benchmark tools (may need manual installation)
sudo dnf install -y iozone bonnie++ dbench
```

Note for RHEL/CentOS users: Some benchmark tools (iozone, bonnie++, dbench) may not be available in standard repositories. If installation fails, you can:

1. **Enable EPEL repository** (recommended):

```
sudo dnf install -y epel-release
sudo dnf install -y iozone bonnie++ dbench
```

2. **Install from source** (if EPEL doesn't have them):

   IOzone:

```
wget http://www.iozone.org/src/current/iozone3_506.tar
tar xf iozone3_506.tar
cd iozone3_506/src/current
make linux
sudo cp iozone /usr/local/bin/
```

   - Official site: http://www.iozone.org/
   - GitHub mirror: https://github.com/chaos/iozone

   Bonnie++:

```
wget https://www.coker.com.au/bonnie++/bonnie++-2.00a.tgz
tar xzf bonnie++-2.00a.tgz
cd bonnie++-2.00a
./configure
make
sudo make install
```

   - Official site: https://www.coker.com.au/bonnie++/
   - GitHub: https://github.com/sbates130272/bonnie

dbench:

```
git clone https://github.com/sahlberg/dbench.git
cd dbench
./autogen.sh
./configure
make
sudo make install
```

- GitHub: https://github.com/sahlberg/dbench

3. **Run tests without missing tools:**

```
# Skip specific tests if tools are missing
python3 runtest.py --mount-path /mnt/nfs1 --skip-iozone --skip-bonnie --skip-dbench
```

Python Dependencies

```
pip3 install --user -r requirements.txt
```

Installation

```
git clone https://github.com/skmprabhu252/nfs-benchmark-suite.git
cd nfs-benchmark-suite
./setup_and_verify.sh --auto
```

Quick Start

```
# Run quick test (~15 minutes)
python3 runtest.py --mount-path /mnt/nfs1 --quick-test

# Generate HTML report
python3 generate_html_report.py nfs_performance_test_*.json
```

Usage

Basic Usage

```
# Quick test (validation, ~15 minutes)
python3 runtest.py --mount-path /mnt/nfs1 --quick-test

# Long test (benchmark, 4-8 hours)
python3 runtest.py --mount-path /mnt/nfs1 --long-test

# Default test (balanced, ~30 minutes)
python3 runtest.py --mount-path /mnt/nfs1

# Custom configuration
python3 runtest.py --mount-path /mnt/nfs1 --config my_config.yaml

# Verbose output
python3 runtest.py --mount-path /mnt/nfs1 --verbose
```

Skip Specific Tests

```
python3 runtest.py --mount-path /mnt/nfs1 --skip-dd
python3 runtest.py --mount-path /mnt/nfs1 --skip-fio
```

```
python3 runtest.py --mount-path /mnt/nfs1 --skip-iozone
python3 runtest.py --mount-path /mnt/nfs1 --skip-bonnie
python3 runtest.py --mount-path /mnt/nfs1 --skip-dbench
```

Common Scenarios

```
# Test different mount options
mount -t nfs -o vers=4.2,rsize=1048576,wsize=1048576 server:/export /mnt/nfs1
python3 runtest.py --mount-path /mnt/nfs1

# Compare NFSv3 vs NFSv4
mount -t nfs -o vers=3 server:/export /mnt/nfs3
mount -t nfs -o vers=4.2 server:/export /mnt/nfs4
python3 runtest.py --mount-path /mnt/nfs3
python3 runtest.py --mount-path /mnt/nfs4

# Disable history tracking (for testing)
python3 runtest.py --mount-path /mnt/nfs1 --no-save-history
```

Configuration

Configuration Files

The suite includes three pre-configured test profiles:

1. **config/config_quick_test.yaml** - Quick validation test (~15 minutes)
2. **config/config_long_test.yaml** - Production benchmark (4-8 hours)

README

Custom Test Configuration

```
# Copy default config
cp config/test_config.yaml my_config.yaml

# Edit as needed
vim my_config.yaml

# Run with custom config
python3 runtest.py --mount-path /mnt/nfs1 --config my_config.yaml
```

Example Configuration

```yaml
dd_tests:
  sequential_write_direct:
    enabled: true
    block_size: "1M"
    count: 100000

fio_tests:
  sequential_write:
    enabled: true
    bs: "1M"
    size: "4G"
    numjobs: 1
    runtime: 60
```

Understanding Results

Output Files

After running tests, you'll get:

- **JSON file**: `nfs_performance_test_YYYYMMDD_HHMMSS.json` - Raw results

- **Log file**: `nfs_performance_test_YYYYMMDD_HHMMSS.log` - Detailed logs
- **HTML report**: Generate with `python3 generate_html_report.py <json_file>`

Key Metrics

Test Type	What It Measures	Good For	What To Look For
Throughput Tests (DD, FIO Sequential)	MB/s for large file transfers	Evaluating bulk data transfer performance	Close to network bandwidth limit (e.g., ~1,100 MB/s for 10 GbE)
IOPS Tests (FIO Random)	Operations per second for small random I/O	Database and application workloads	Consistent IOPS without high latency
Latency Tests	Response time in milliseconds	Interactive applications	Low and consistent latency (<5ms average)
Metadata Tests (IOzone, Bonnie++)	File operations per second	Applications with many small files	High file creation/deletion rates
Client Simulation (dbench)	Multi-client throughput and latency	Understanding concurrent user performance	Linear scaling up to 4-8 clients

Performance Baselines (10 GbE)

Test	Minimum	Good	Excellent
Sequential Read	500 MB/s	800 MB/s	1,100 MB/s
Sequential Write	350 MB/s	600 MB/s	900 MB/s
Random Read (4K)	8K IOPS	20K IOPS	40K IOPS

Test	Minimum	Good	Excellent
Random Write (4K)	5K IOPS	15K IOPS	30K IOPS
Latency (avg)	<10ms	<5ms	<2ms
Metadata ops/sec	1K ops/s	5K ops/s	10K ops/s

Note: Baselines vary by network speed

- 1 GbE: ~100 MB/s sequential
- 10 GbE: ~1,000 MB/s sequential
- 25 GbE: ~2,500 MB/s sequential
- 100 GbE: ~10,000 MB/s sequential

Historical Comparison

The tool automatically tracks performance over time:

- ✅ **Improved**: Performance increased (green)
- ❌ **Regressed**: Performance decreased >10% (red)
- ⚠️ **Warning**: Performance decreased 5-10% (yellow)
- ➡️ **Stable**: Performance within ±5% (gray)

Best Practices

Testing Guidelines

1. **Run During Maintenance Windows**: Tests generate significant load (especially long tests)
2. **Avoid Concurrent Workloads**: Ensure no other processes using the NFS mount
3. **Run Multiple Iterations**: Run 3-5 times for consistent results

4. **Compare Apples to Apples**: Use same test profile when comparing configurations
5. **Check Network First**: Use `iperf3` to verify network performance before blaming NFS
6. **Monitor Server**: Check NFS server CPU, memory, and storage during tests
7. **Document Configuration**: Record mount options, NFS version, and server details

For Quick Tests

- Run before making any configuration changes
- Use as a smoke test after NFS mount changes
- Integrate into CI/CD pipelines for automated validation
- Run multiple times to verify consistency

For Long Tests

- Schedule during maintenance windows
- Ensure no other workloads on the NFS mount
- Monitor NFS server resources during test
- Run 2-3 times for statistical validation
- Document all configuration details
- Compare results against vendor specifications
- Use results for capacity planning decisions

Recommended NFS Mount Options

```
# NFSv4.2 (recommended for best performance)
mount -t nfs -o vers=4.2,rsize=1048576,wsize=1048576,hard,async,noatime server:/export /mnt/nfs1

# NFSv3 (for comparison or compatibility)
mount -t nfs -o vers=3,rsize=1048576,wsize=1048576,hard,async,noatime server:/export /mnt/nfs1
```

Key Mount Options:

Option	Purpose	Impact
`vers=4.2`	Use NFSv4.2 protocol	10-30% faster than NFSv3
`rsize/wsize=1048576`	1MB read/write buffer	Optimal for 10 GbE networks
`hard`	Retry indefinitely on failure	Prevents data loss (recommended for production)
`async`	Better write performance	Use `sync` for critical data requiring immediate persistence
`noatime`	Don't update file access times	Reduces metadata operations by ~20%

Performance Tips by Network Speed:

Network Speed	Recommended rsize/wsize	Notes
1 GbE	`262144` (256KB)	Standard gigabit
10 GbE	`1048576` (1MB)	Most common enterprise
25+ GbE	`1048576` (1MB)	High-performance networks

Workload-Specific Tips:

Workload Type	Recommendation
Latency-sensitive apps	Consider `sync` instead of `async`
Read-heavy workloads	Increase client cache with `actimeo=600`
Write-heavy workloads	Use `async` for better performance

Troubleshooting

Common Issues

1. Permission Denied

```
# Test write access
touch /mnt/nfs1/test && rm /mnt/nfs1/test

# Check mount options
mount | grep nfs

# Fix: Ensure export allows read/write
# On NFS server: /etc/exports
# /export client_ip(rw,sync,no_root_squash)
```

2. Low Performance

```
# Check network connectivity
ping -c 100 nfs-server
iperf3 -c nfs-server

# Check mount options
mount | grep nfs

# Common fixes:
# - Increase rsize/wsize to 1048576
# - Use NFSv4.2 instead of NFSv3
# - Enable async (if data loss acceptable)
# - Check storage backend performance
```

3. Tool Not Found

```
# Ubuntu/Debian
sudo apt-get install fio iozone bonnie++ dbench libnsl2

# RHEL/CentOS - try EPEL first
sudo dnf install -y epel-release
sudo dnf install fio iozone bonnie++ dbench libnsl
```

```
# If tools not available in repos, see Requirements section for manual installation
```

4. Test Hangs or Times Out

```
# Check NFS server status
showmount -e nfs-server

# Check for NFS errors
dmesg | grep -i nfs

# The tool has built-in timeout protection
# Tests will automatically fail after timeout period
```

5. Inconsistent Results

```
# Run multiple iterations
for i in {1..3}; do
  python3 runtest.py --mount-path /mnt/nfs1 --quick-test
done

# Avoid concurrent workloads
# Run during maintenance windows
# Ensure no other processes using NFS mount
```

6. Disk Space Errors

```
# Check available space
df -h /mnt/nfs1
```